

```

import matplotlib.pyplot as plt
import numpy as np
from scipy.signal import bilinear, butter, cheby1, freqz, lfilter

def design_butterworth_filter(filter_order, cutoff_frequency, sampling_frequency):
    # Normalize the cutoff frequency (cutoff_frequency / Nyquist frequency)
    nyquist_frequency = sampling_frequency / 2
    normalized_cutoff = cutoff_frequency / nyquist_frequency

    # Design the analog Butterworth filter
    analog_b, analog_a = butter(
        filter_order, normalized_cutoff, analog=False, btype="low"
    )

    # Perform the bilinear transformation
    digital_b, digital_a = bilinear(analog_b, analog_a, sampling_frequency)

    return digital_b, digital_a

def design_chebyshev_filter(filter_order, cutoff_frequency, sampling_frequency, ripple):
    # Normalize the cutoff frequency
    nyquist_frequency = sampling_frequency / 2
    normalized_cutoff = cutoff_frequency / nyquist_frequency

    # Design the analog Chebyshev filter
    analog_b, analog_a = cheby1(
        filter_order, ripple, normalized_cutoff, analog=False, btype="low"
    )

    # Perform the bilinear transformation
    digital_b, digital_a = bilinear(analog_b, analog_a, sampling_frequency)

    return digital_b, digital_a

def plot_filter_response(digital_b, digital_a, sampling_frequency):
    # Compute the frequency response of the filter
    frequency, magnitude_response = freqz(digital_b, digital_a, fs=sampling_frequency)

    # Plot the magnitude response
    plt.figure(figsize=(10, 6))
    plt.plot(frequency, np.abs(magnitude_response))
    plt.title("Filter Magnitude Response")
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude")
    plt.grid(True)
    plt.show()

    # Compute and plot the impulse response
    impulse = np.zeros(100)
    impulse[0] = 1 # Create a unit impulse
    impulse_response = lfilter(digital_b, digital_a, impulse)

    plt.figure(figsize=(10, 6))
    plt.plot(impulse_response)
    plt.title("Filter Impulse Response")
    plt.xlabel("Samples")
    plt.ylabel("Amplitude")
    plt.grid(True)
    plt.show()

# Specify the desired filter specifications
filter_order = 4 # Filter order
cutoff_frequency = 1000 # Cutoff frequency in Hz
sampling_frequency = 8000 # Sampling frequency in Hz
ripple = 0.5 # Ripple factor for Chebyshev filter

# Design the Butterworth filter
digital_b, digital_a = design_butterworth_filter(
    filter_order, cutoff_frequency, sampling_frequency
)

# Plot the Butterworth filter's magnitude response and impulse response
plot_filter_response(digital_b, digital_a, sampling_frequency)

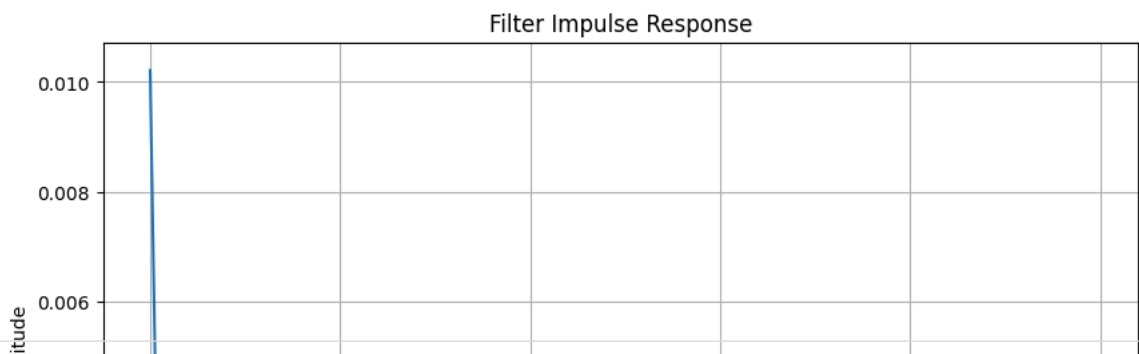
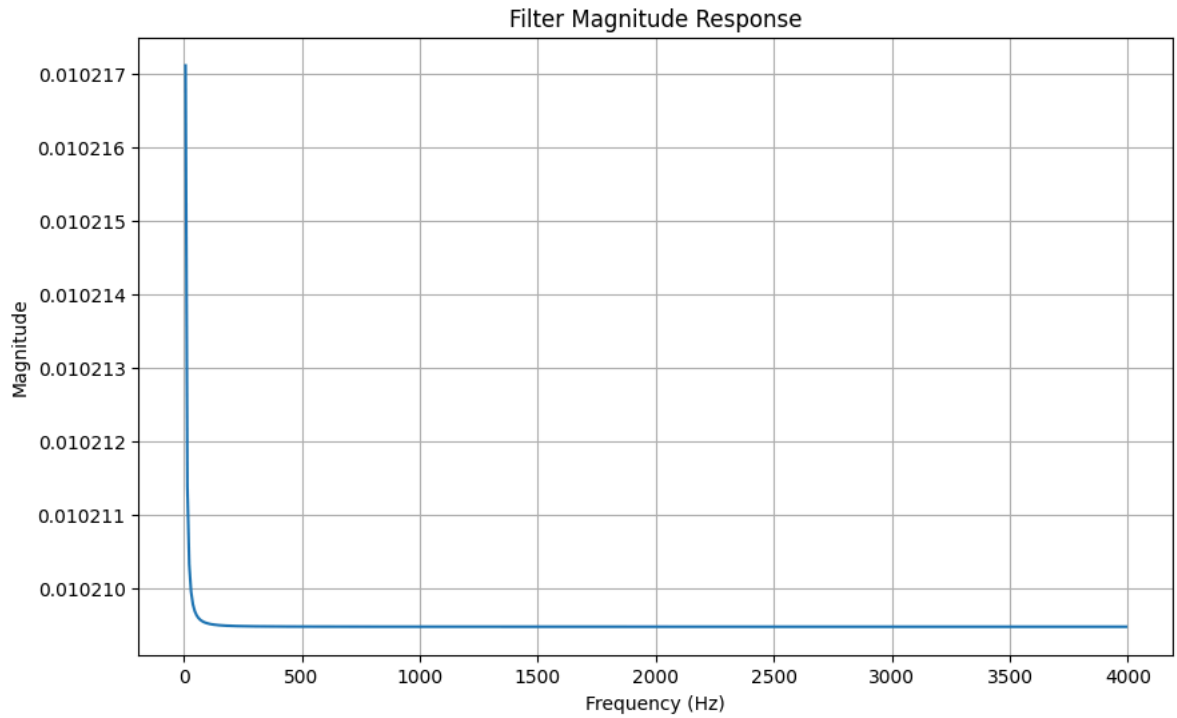
# Design the Chebyshev filter
digital_b, digital_a = design_chebyshev_filter(
    filter_order, cutoff_frequency, sampling_frequency, ripple
)

```

```
# Plot the chebyshev filter's magnitude response and impulse response
plot_filter_response(digital_b, digital_a, sampling_frequency)

# Save the filter coefficients (optional)
filter_path = "filter_coefficients.txt"
np.savetxt(filter_path, np.vstack((digital_b, digital_a)), delimiter=",")
print(f"Filter coefficients saved at: {filter_path}")
```

```
/usr/local/lib/python3.12/dist-packages/scipy/signal/_filter_design.py:564: RuntimeWarning: divide by zero encount  
h = (_pu.npp_polyval(zm1, b, tensor=False, xp=xp) /  
/usr/local/lib/python3.12/dist-packages/scipy/signal/_filter_design.py:564: RuntimeWarning: invalid value encounte  
h = (_pu.npp_polyval(zm1, b, tensor=False, xp=xp) /
```



Start coding or [generate](#) with AI.

0.004

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

